Evidence-Grounded Research Paper Fact-Checking Pipeline

Submitted by

Mega Sri B U D

2403917710422146

Akchaya B N

2403917710422113

ABSTRACT

Research papers contain numerous technical claims involving model architectures, parameter counts, computational requirements, experimental results, and comparisons with existing approaches. Manually verifying these claims against the original research papers is time-consuming, particularly when the source documents are lengthy. The **Evidence-Grounded Research Paper Fact-Checking Pipeline** is designed to automate this process by retrieving relevant evidence from research papers and using that evidence to evaluate individual claims.

The proposed pipeline begins with PDF research papers as input. The textual content is extracted from the PDFs and divided into smaller chunks. These chunks are converted into semantic embeddings using SentenceTransformers and indexed using FAISS for efficient similarity-based retrieval. Claims can then be evaluated by retrieving the most relevant portions of the source paper. The retrieved passages are used as evidence for subsequent verification.

The project provides two verification modes. The first is a **Gemini-based verification mode**, which uses an LLM-based verifier–critic–judge workflow orchestrated using LangGraph. The second is a **local retrieval-based fallback mode**, which operates without consuming Gemini API quota. The local mode is heuristic and retrieval-based and therefore is not equivalent to LLM-based verification.

The project contains research-paper data for LoRA, QLoRA, LoRA+, DistilBERT, and ALBERT. The supplied ground-truth dataset contains **22 manually verified claims**, consisting of 4 LoRA claims, 5 QLoRA claims, 3 LoRA+ claims, 5 DistilBERT claims, and 5 ALBERT claims. The ground-truth verdict for all 22 claims is **SUPPORTED**.

A demonstrated local evaluation was performed on five LoRA claims. Four claims were classified as supported, zero as contradicted, and one as unaddressed. The reported confidence values were 1.00, 1.00, 0.96, 0.40, and 0.92. The project test suite also completed with **4 passed tests and 1 warning in 14.13 seconds**. The warning was associated with a Google GenAI SDK deprecation notice and did not indicate a pipeline failure.

KEYWORDS

Research Paper Fact-Checking, Evidence Retrieval, Semantic Search, SentenceTransformers, FAISS, LangGraph, Gemini, Claim Verification, Natural Language Processing, Large Language Models, Vector Database, Evidence-Grounded AI

1. INTRODUCTION

The increasing volume of scientific and technical literature has created a growing need for efficient methods of examining and verifying information contained in research papers. Modern research papers frequently report numerical results, architectural innovations, parameter reductions, computational requirements, and comparisons with previous approaches. Verifying such statements manually requires locating the corresponding evidence in the source document and determining whether the evidence actually supports the claim.

Traditional keyword-based search is insufficient for this purpose because the wording of a claim may differ from the wording used in the original paper. Semantic retrieval provides a more suitable approach because it represents text using numerical embeddings and retrieves passages according to semantic similarity.

The **Evidence-Grounded Research Paper Fact-Checking Pipeline** combines document processing, semantic representation, vector retrieval, and structured verification into a single workflow. Research papers are first processed into searchable textual chunks. SentenceTransformers generates semantic representations of these chunks, while FAISS provides efficient vector similarity search.

The retrieved evidence is then passed through the verification architecture. The project supports a Gemini-based workflow organized using LangGraph and a local retrieval-based fallback mode. The latter is particularly useful when Gemini API quota is unavailable.

An important characteristic of the system is its distinction between insufficient evidence and contradiction. If the available retrieved evidence does not sufficiently establish a claim, the system uses the verdict **unaddressed** rather than incorrectly declaring the claim false.

2. PROBLEM STATEMENT

Research papers contain large quantities of technical information that must often be verified before being used in reports, presentations, reviews, or further research. Manual verification involves searching through papers, locating relevant passages, comparing the wording and numerical details of claims, and determining whether the evidence supports the statement.

The problem addressed by this project is therefore:

To develop an evidence-grounded automated pipeline that retrieves relevant information from research papers and verifies technical claims against the retrieved source evidence while distinguishing supported, contradicted, and insufficiently evidenced claims.

The system should reduce the effort required to locate relevant evidence while maintaining a clear connection between each verdict and the source material.

3. OBJECTIVES

The major objectives of the project are:

1. To extract textual information from research-paper PDFs.

2. To divide extracted documents into retrieval-friendly chunks.
3. To generate semantic embeddings for document chunks.
4. To store the embeddings using FAISS.
5. To retrieve semantically relevant evidence for claims.
6. To extract and process research-paper claims.
7. To perform evidence-based claim verification.
8. To implement a verifier–critic–judge workflow.
9. To orchestrate the verification workflow using LangGraph.
10. To provide Gemini-based LLM verification.
11. To provide a local fallback mode when Gemini quota is unavailable.
12. To generate structured fact-checking reports.
13. To evaluate the system using manually verified ground-truth claims.
14. To provide a distinction between supported, contradicted, and unaddressed claims.

4. SCOPE OF THE PROJECT

The project focuses on fact-checking claims against evidence contained within research papers.

The scope includes:

- PDF processing
- Text extraction
- Text chunking
- Semantic embeddings
- FAISS vector indexing
- Semantic retrieval
- Claim extraction
- Evidence-based verification
- Critic evaluation
- Final judging
- Gemini-based verification
- Local heuristic verification
- Automated testing

- Structured result generation
- Ground-truth-based evaluation

The current project contains source-paper material for:

- LoRA
- QLoRA
- LoRA+
- DistilBERT
- ALBERT

The system is primarily designed for **source-grounded verification**, where evidence is retrieved from the indexed research-paper collection. It is not demonstrated as a web-scale fact-checking system or as a general-purpose truth verification engine.

5. SYSTEM REQUIREMENTS

5.1 Software Requirements

Component	Technology
Programming Language	Python
PDF Processing	PyMuPDF
Embedding Generation	SentenceTransformers
Vector Search	FAISS
Numerical Processing	NumPy
Workflow Orchestration	LangGraph
LLM Integration	Google GenAI
Environment Configuration	python-dotenv
Testing	pytest
Output Formats	JSON, Markdown
Development Environment	Visual Studio Code

5.2 Python Dependencies

The project uses the following packages:

google-genai

python-dotenv

langgraph

sentence-transformers

faiss-cpu

numpy

pymupdf

pytest

5.3 Hardware and Operating Environment

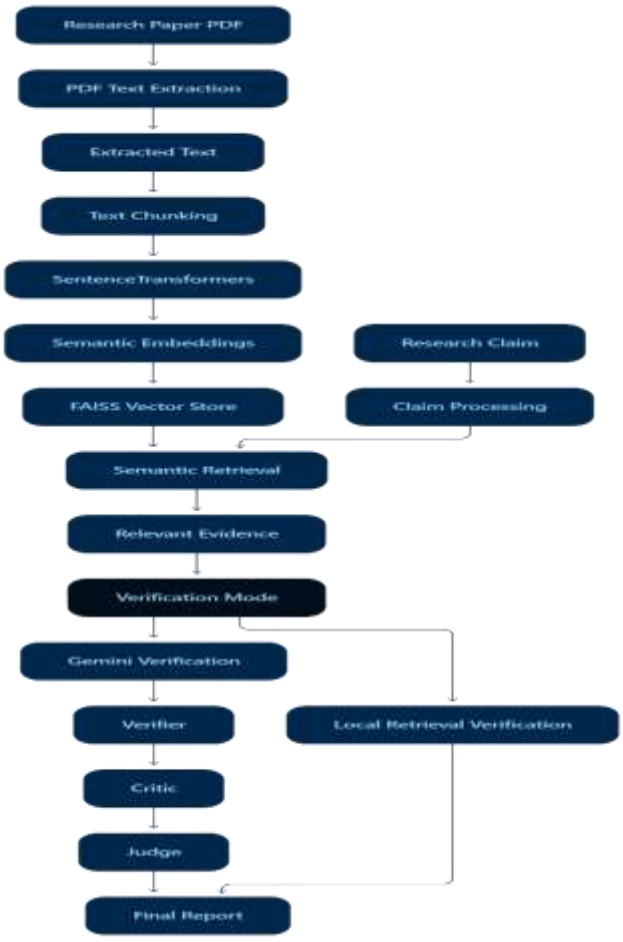
The project was demonstrated in a Windows development environment using Python.

The screenshots show execution from a Windows Python environment and development through Visual Studio Code.

6. SYSTEM ARCHITECTURE

The complete system consists of a document indexing pipeline and a claim verification pipeline.

Figure 1: Overall System Architecture



7. METHODOLOGY

7.1 PDF Text Extraction

The first stage of the pipeline accepts research papers in PDF format.

The project contains research-paper PDFs including:

albert.pdf

distilbert.pdf

lora_plus.pdf

lora.pdf

qlora.pdf

PyMuPDF is used for PDF processing and text extraction. The extracted text is stored separately so that subsequent stages do not need to repeatedly process the original PDF.

The project contains extracted text files corresponding to the research papers.

7.2 Text Chunking

After extraction, the document text is divided into smaller chunks.

This is necessary because a complete research paper may be too large to efficiently process as a single retrieval unit. Smaller chunks allow the retrieval system to identify the specific portions of a paper that are most relevant to a claim.

The project contains chunk files such as:

albert_chunks.txt

distilbert_chunks.txt

lora_chunks.txt

lora_plus_chunks.txt

qlora_chunks.txt

The chunking stage therefore converts a long document into a collection of searchable evidence units.

7.3 Sentence Embeddings

The text chunks are converted into semantic vector representations using **SentenceTransformers**.

Instead of representing a passage only through individual keywords, an embedding captures its semantic representation in numerical form.

The process is:

Text Chunk

↓

SentenceTransformer

↓

Numerical Vector

These vectors are subsequently indexed for retrieval.

7.4 FAISS Vector Retrieval

The generated embeddings are stored in a FAISS vector index.

The project contains:

data/vector_store/

├── documents.pkl

└── faiss_index.bin

When a claim is submitted, its semantic representation can be compared with the indexed document representations.

Claim

↓

Claim Embedding

↓

FAISS Similarity Search

↓

Relevant Document Chunks

The retrieved chunks act as the evidence supplied to the verification stage.

7.5 Claim Extraction

The project contains a dedicated claim_extractor.py module.

Claim extraction identifies statements that are suitable for verification. These claims can include:

- numerical claims,
- model-performance claims,

- architectural claims,
- computational claims,
- comparative claims.

The extracted claims are subsequently evaluated against retrieved evidence.

7.6 Evidence-Based Verification

The central principle of the project is that verification should be grounded in evidence retrieved from the source paper.

A claim can receive one of the following outcomes:

Verdict	Meaning
Supported	Retrieved evidence sufficiently supports the claim
Contradicted	Retrieved evidence conflicts with the claim
Unaddressed	Retrieved evidence is insufficient to establish the claim

The **unaddressed** category is important because a failure to retrieve sufficient evidence does not necessarily mean that the claim is false.

8. VERIFIER–CRITIC–JUDGE WORKFLOW

The LLM-based verification architecture uses three conceptual decision stages.



Figure 2. Verifier–critic–judge workflow used for structured claim verification.

8.1 Verifier

The verifier examines the claim together with the retrieved evidence and generates an initial assessment.

Its purpose is to determine whether the evidence supports the factual content of the claim.

8.2 Critic

The critic provides an independent assessment of the initial verification.

The use of a separate critic stage is intended to identify weaknesses or inconsistencies in the initial decision.

8.3 Judge

The judge represents the final decision stage.

It considers the verification and critique before producing the final verdict that is included in the fact-checking report.

9. LOCAL FALLBACK MODE

The project includes a local retrieval-based fallback mode.

The local mode can be executed using:

python src/run_fact_checker.py

python src/create_report.py

The generated results are stored as:

results/lora_local_results.json

results/lora_report.md

The main advantage of this mode is that it does not consume Gemini API quota.

However, the local mode must be interpreted correctly. It is a **heuristic retrieval-based verification approach**. It is not equivalent to LLM-based verification using Gemini.

Therefore, a high local confidence value should not automatically be interpreted as equivalent to a high-quality LLM reasoning result.

10. TECHNOLOGIES AND LIBRARIES USED

Technology	Role in the Project
Python	Main programming language
PyMuPDF	PDF text extraction
SentenceTransformers	Semantic embedding generation
FAISS	Vector similarity retrieval
NumPy	Numerical/vector processing
LangGraph	Verification workflow orchestration
Google GenAI	Gemini integration
python-dotenv	Environment configuration
pytest	Automated testing
JSON	Structured result storage

Markdown	Generated report format

11. PROJECT DIRECTORY STRUCTURE

The project is organized into data, source-code, testing, and result-generation components.

```
PromptEngineering/
│
├── .pytest_cache/
│
├── data/
│   ├── abstracts/
│   │   ├── ard_lora_abstract.txt
│   │   ├── effilora_abstract.txt
│   │   └── lora_xs_abstract.txt
│   │
│   ├── chunks/
│   │   ├── albert_chunks.txt
│   │   ├── distilbert_chunks.txt
│   │   ├── lora_chunks.txt
│   │   ├── lora_plus_chunks.txt
│   │   └── qlora_chunks.txt
│   │
│   ├── extracted_text/
│   │   ├── albert.txt
│   │   ├── distilbert.txt
│   │   ├── lora_plus.txt
│   │   ├── lora.txt
│   │   └── qlora.txt
│   │
│   ├── papers/
│   │   ├── albert.pdf
│   │   ├── distilbert.pdf
│   │   ├── lora_plus.pdf
│   │   ├── lora.pdf
│   │   └── qlora.pdf
│   │
│   ├── vector_store/
│   │   ├── documents.pkl
│   │   └── faiss_index.bin
│   │
│   └── ground_truth.csv
│
├── results/
│   ├── lora_claims.json
│   ├── lora_local_results.json
```

```
│   └── lora_report.md
│
├── src/
│   ├── chunker.py
│   ├── claim_extractor.py
│   ├── create_report.py
│   ├── critic.py
│   ├── graph.py
│   ├── judge.py
│   ├── pdf_processor.py
│   ├── prompts.py
│   ├── run_fact_checker.py
│   ├── vector_store.py
│   └── verifier.py
│
├── tests/
│   └── test_local_pipeline.py
│
├── .env
├── README.md
└── test_retrieval.py
```

The .env file is part of the project configuration but its contents are intentionally excluded from this report for security reasons.

12. IMPLEMENTATION DETAILS

The project follows a modular implementation approach.

Source File	Function
pdf_processor.py	PDF processing and text extraction
chunker.py	Document chunking
vector_store.py	Vector-store creation and retrieval
claim_extractor.py	Claim extraction
verifier.py	Verification stage
critic.py	Critical assessment
judge.py	Final judgment
graph.py	LangGraph workflow
run_fact_checker.py	Local fact-checking execution
create_report.py	Report generation

prompts.py	Prompt-related definitions
test_local_pipeline.py	Local pipeline testing
test_retrieval.py	Retrieval testing

The modular design separates document processing, retrieval, verification, orchestration, and reporting.

13. COMPLETE WORKFLOW

The complete workflow can be summarized as follows:

Stage 1 — Input

A research paper is supplied as a PDF.

Stage 2 — Extraction

PyMuPDF extracts the textual content from the PDF.

Stage 3 — Chunking

The extracted text is divided into smaller chunks.

Stage 4 — Embedding

SentenceTransformers generates semantic embeddings for the chunks.

Stage 5 — Indexing

The embeddings are stored in a FAISS vector index.

Stage 6 — Claim Processing

Claims are extracted or supplied for verification.

Stage 7 — Retrieval

The claim is compared against the indexed chunks and relevant evidence is retrieved.

Stage 8 — Verification

The retrieved evidence is used to determine whether the claim is supported, contradicted, or unaddressed.

Stage 9 — Critical Review

In the LLM workflow, the critic independently examines the initial verification.

Stage 10 — Final Judgment

The judge produces the final verification decision.

Stage 11 — Report Generation

The claim, verdict, confidence, reasoning, and evidence are recorded in the generated result.

14. EXPERIMENTAL SETUP

The project contains research-paper material for five papers:

Paper	Number of Ground-Truth Claims
LoRA	4
QLoRA	5
LoRA+	3
DistilBERT	5
ALBERT	5
Total	**22**

The supplied ground_truth.csv contains 22 manually verified claims. All 22 ground-truth entries are marked **SUPPORTED**.

The ground-truth claims cover different categories of technical statements, including:

- parameter reduction,
- GPU memory requirements,
- model performance,
- model architecture,
- optimization techniques,
- training speed,
- benchmark performance,
- knowledge distillation,
- parameter efficiency.

The local fact-checking demonstration specifically evaluates five LoRA claims.

15. GROUND-TRUTH DATASET

The ground-truth dataset contains manually verified claims and corresponding source information.

Each record contains fields for:

- paper identifier,
- paper title,
- claim identifier,

- claim,
- ground-truth verdict,
- ground-truth evidence,
- source section,
- verification notes.

The dataset contains **22 claims**, all marked as SUPPORTED.

Distribution

Research Paper	Claims	Ground-Truth Supported
LoRA	4	4
QLoRA	5	5
LoRA+	3	3
DistilBERT	5	5
ALBERT	5	5
Total	**22**	**22**

The ground-truth dataset therefore establishes a manually verified reference set for future system evaluation.

It should not, however, be interpreted as a measured model accuracy result because the supplied information does not provide predictions for all 22 claims from the same evaluation run.

16. RESULTS AND EVALUATION

16.1 Local LoRA Evaluation

The demonstrated local-mode evaluation contains five claims.

Metric	Result
Total claims	5
Supported	4
Contradicted	0
Unaddressed	1
Verification mode	Local retrieval

Based on the five supplied local results:

- Supported: **80%**

- Contradicted: **0%**
- Unaddressed: **20%**

These percentages describe the observed verdict distribution for the supplied five-claim local experiment. They should not be interpreted as a general accuracy score.

17. CLAIM-LEVEL RESULTS

Claim	Verdict	Confidence
1. Full fine-tuning becomes less feasible as models become larger	Supported	1.00
2. Independent GPT-3 175B fine-tuned instances are prohibitively expensive	Supported	1.00
3. LoRA freezes pretrained weights and introduces trainable rank-decomposition matrices	Supported	0.96
4. LoRA reduces trainable parameters by 10,000× and GPU memory by 3×	Unaddressed	0.40
5. LoRA performs on-par or better than fine-tuning across the stated models	Supported	0.92

17.1 Claim 1

Claim:
As larger models are pretrained, full fine-tuning, which retrains all model parameters, becomes less feasible.

Verdict: Supported

Confidence: 1.00

The retrieved LoRA evidence directly contains the corresponding statement regarding the increasing difficulty of full fine-tuning as model size increases.

17.2 Claim 2

Claim:
Deploying independent fine-tuned GPT-3 175B instances for individual tasks is prohibitively expensive.

Verdict: Supported

Confidence: 1.00

The retrieved evidence contains the GPT-3 175B example and directly describes the deployment cost issue.

17.3 Claim 3

Claim:
LoRA freezes pretrained model weights and introduces trainable rank-decomposition matrices into Transformer layers.

Verdict: Supported

Confidence: 0.96

The retrieved evidence directly describes the LoRA architecture and its strategy of freezing pretrained weights while training low-rank matrices.

17.4 Claim 4

Claim:
Compared with GPT-3 175B fine-tuned with Adam, LoRA can reduce the number of trainable parameters by 10,000 times and GPU memory requirements by 3 times.

Verdict: Unaddressed

Confidence: 0.40

The retrieved evidence contained related LoRA and GPT-3 information but did not sufficiently establish all the numerical details required by this claim.

The system therefore classified the claim as **unaddressed**, rather than contradicted.

This illustrates an important design principle of the project:

Insufficient evidence is not automatically treated as evidence of falsity.

17.5 Claim 5

Claim:
LoRA performs on-par or better than fine-tuning on RoBERTa, DeBERTa, GPT-2, and GPT-3 while using fewer trainable parameters, achieving higher training throughput, and introducing no additional inference latency compared with adapters.

Verdict: Supported

Confidence: 0.92

The retrieved evidence directly contains the corresponding performance comparison.

18. VERDICT SUMMARY

Total Claims : 5

Supported : 4

Contradicted : 0

Unaddressed : 1

Verification Mode : Local Retrieval

Observed Distribution

Supported ████████████████ 80%

Unaddressed ████ 20%

Contradicted 0%

The local evaluation demonstrates that the system can identify evidence supporting claims and can also avoid making an unsupported contradiction decision when retrieved evidence is insufficient.

19. TEST RESULTS

The supplied test execution produced:

4 passed, 1 warning in 14.13s

Therefore:

Test Metric	Result
Tests executed	4
Passed	4
Failed	0
Warnings	1
Execution time	14.13 seconds

The result indicates that all four executed tests passed.

19.1 SDK Warning

The warning shown during execution originates from the Google GenAI Python package and concerns the deprecated _UnionGenericAlias type.

The warning is a **Python SDK/dependency deprecation warning**. It is not evidence of a failed pipeline execution.

Therefore, the test result remains:

4 passed, 1 warning in 14.13 seconds.

20. GEMINI-BASED AND LOCAL VERIFICATION

The system supports two different execution modes.

Characteristic	Gemini Mode	Local Mode
Semantic retrieval	Yes	Yes
FAISS retrieval	Yes	Yes
LLM-based reasoning	Yes	No
Gemini quota required	Yes	No
Verification approach	LLM-based	Heuristic/retrieval-based
Demonstrated results	Not available	Available
Use when quota unavailable	No	Yes

Gemini Mode

The Gemini mode integrates the Google GenAI API into the verification workflow.

However, during the reported experiment, the available Gemini API quota had been exhausted. Consequently, a Gemini execution could not be completed for the reported evaluation.

Therefore, no Gemini accuracy, verdict distribution, confidence statistics, or execution performance is claimed in this report.

Local Mode

The local mode was successfully used for the demonstrated LoRA evaluation.

It does not require Gemini quota and provides a fallback mechanism for evidence retrieval and heuristic verification.

The local results reported in this project are therefore explicitly identified as **local retrieval results**.

The two modes should not be treated as interchangeable. The local mode does not reproduce the full reasoning capabilities of a Gemini-based LLM verifier.

21. LIMITATIONS AND MITIGATIONS

Limitation	Impact	Mitigation
Gemini API quota exhaustion	Gemini verification could not be demonstrated during the reported experiment	Use local retrieval mode when quota is unavailable
Local verification is heuristic	It does not provide the same reasoning capabilities as an LLM	Clearly distinguish local results from Gemini results

Retrieval quality affects verification	Relevant evidence may not always be retrieved	Improve chunking, embedding and retrieval strategies
Limited demonstrated local evaluation	Five claims were used in the reported local result	Expand evaluation to the full ground-truth dataset
No contradicted claim in the demonstrated local results	Contradiction detection is not demonstrated by the five-claim experiment	Add verified contradictory test cases
Ground truth contains supported claims only	It does not independently test contradiction classification	Extend ground truth with contradicted and unaddressed examples
Exact embedding model is not documented in the supplied information	Exact reproduction of embedding generation is incomplete	Record the embedding model in future experiments
SDK deprecation warning	Produces a warning during execution	Update affected dependencies/code
Source-bound verification	Claims outside the indexed evidence may remain unaddressed	Expand the indexed research corpus

22. SECURITY AND ETHICAL CONSIDERATIONS

22.1 API Credential Protection

The Gemini API key is stored through environment configuration and should not be included in source code or reports.

The .env file is therefore excluded from the technical documentation.

22.2 Protection of Personal Information

Personal paths, email addresses, credentials, and other unrelated personal information should not be included in screenshots or the final report.

22.3 Evidence-Based Decisions

The system should associate verification decisions with retrieved evidence rather than presenting unsupported conclusions as facts.

22.4 Avoiding False Contradictions

The use of the unaddressed verdict prevents a lack of retrieved evidence from automatically being interpreted as contradiction.

This is especially important in scientific fact checking, where failure to locate evidence is not necessarily evidence that a statement is false.

22.5 Responsible Use of LLMs

Gemini-based verification should be interpreted as an automated assistance mechanism. LLM outputs should remain grounded in the retrieved source evidence and should not be treated as infallible scientific conclusions.

23. FUTURE ENHANCEMENTS

23.1 Expanded Evaluation

The existing 22-claim ground-truth dataset can be used for a larger systematic evaluation covering all five research papers.

23.2 Contradiction Dataset

Additional deliberately contradicted claims can be introduced to evaluate the system's ability to distinguish contradiction from insufficient evidence.

23.3 Improved Claim Decomposition

Compound claims can be separated into atomic claims so that each individual numerical or factual component can be independently verified.

23.4 Hybrid Retrieval

Dense vector retrieval can be combined with traditional keyword or BM25 retrieval to improve evidence recall.

23.5 Reranking

A reranking model can be introduced after FAISS retrieval to improve the ordering of candidate evidence passages.

23.6 Page-Level Citations

The system can be enhanced to associate every retrieved passage with its exact page number and section in the source paper.

23.7 Confidence Calibration

Confidence values can be compared against actual ground-truth outcomes to determine whether confidence scores are appropriately calibrated.

23.8 Gemini vs Local Comparison

The same set of claims can be evaluated using both Gemini and local verification, allowing their performance and behavior to be systematically compared.

23.9 Interactive User Interface

A graphical interface could display:

- submitted claim,
- retrieved evidence,

- verdict,
- confidence,
- reasoning,
- source location.

23.10 Automated Batch Processing

The pipeline could be extended to automatically process large collections of research papers and generate claim-level fact-checking reports.

24. CONCLUSION

The **Evidence-Grounded Research Paper Fact-Checking Pipeline** provides a structured approach to verifying technical claims against evidence contained in research papers. The project integrates PDF text extraction, text chunking, semantic embeddings, FAISS vector retrieval, claim processing, evidence-based verification, and structured reporting.

The architecture additionally supports a verifier–critic–judge workflow through LangGraph and provides two verification modes: a Gemini-based LLM workflow and a local retrieval-based fallback.

The local mode is particularly useful when Gemini API quota is unavailable. It successfully produced a five-claim LoRA evaluation consisting of four supported claims, zero contradicted claims, and one unaddressed claim. The corresponding confidence values were 1.00, 1.00, 0.96, 0.40, and 0.92.

The project also contains a manually verified ground-truth dataset comprising **22 claims** across LoRA, QLoRA, LoRA+, DistilBERT, and ALBERT. All 22 entries in the supplied ground-truth dataset are marked as supported. This dataset provides a foundation for more comprehensive future evaluation.

The demonstrated automated testing produced **four passed tests and one warning in 14.13 seconds**. The warning is a Google GenAI SDK deprecation warning and does not represent a pipeline failure.

The most important methodological distinction demonstrated by the project is between **contradiction and insufficient evidence**. The fourth LoRA claim was classified as unaddressed because the retrieved evidence did not establish all of its details. This avoids treating incomplete retrieval as proof that a claim is false.

Overall, the project establishes a modular foundation for evidence-grounded research-paper fact checking. Further evaluation using the complete ground-truth dataset, contradictory claims, and successful Gemini executions would provide a stronger basis for quantitative comparison of the verification approaches.

25. REFERENCES

Since you selected **not to include external technical references**, this report restricts the reference section to the project materials and source papers used in the implementation.

1. **LoRA: Low-Rank Adaptation of Large Language Models.** Edward Hu, Yelong Shen, Phillip Wallis, Zeyuan Allen-Zhu, Yuanzhi Li, Shean Wang, Lu Wang, and Weizhu Chen. Version 2.
2. **QLoRA Research Paper.** Source paper included in the project dataset.
3. **LoRA+ Research Paper.** Source paper included in the project dataset.
4. **DistilBERT Research Paper.** Source paper included in the project dataset.
5. **ALBERT Research Paper.** Source paper included in the project dataset.
6. **Project Source Code and Generated Results.** Evidence-Grounded Research Paper Fact-Checking Pipeline.